

Predicting Neural Synapse Formation Using Random Forest

Yiming Wang
Department of Physics and Astronomy
Rice University
Houston, USA
yw99@rice.edu

Grace Xu
Department of Economics
Rice University
Houston, USA
gdx1@rice.edu

Abstract—This study employs a Random Forest classifier to predict neural synapse formation from Axonal Dendritic Proximity (ADP) data, enhanced by specialized feature engineering and SMOTE for class imbalance correction. Our approach achieved balanced accuracies of 72.42% and 77.06% on the Kaggle competition’s public and private leaderboards, respectively. Our results shed light on the effectiveness of machine learning techniques to computational neuroscience, emphasizing the potential of predictive modeling in understanding neural connectivity. We also discuss elements of the broader literature relating to the interplay between neuroscience and machine learning.

Index Terms—neuron, synapse, machine learning, Random Forest, feature engineering, neuroscience

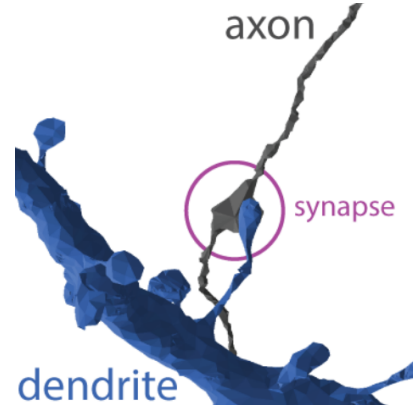


Fig. 1. Synapse (in the purple circle) at the conjunction of the axon (black) and the dendrite (blue). Figure taken from document on Kaggle.

I. INTRODUCTION

Neurons, the fundamental units of the brain and nervous system, communicate with each other through a complex and intricate process. This communication is facilitated by specialized structures known as synapses. A synapse is the junction where the axon terminal of one neuron (the pre-synaptic neuron) meets the dendrite of another (the post-synaptic neuron) [1]. A schematic of synapse is shown in Fig.1. It is at these synapses that neurons transmit signals, playing a critical role in the relay of information throughout the nervous system.

The formation of a synapse begins with an axonal dendritic proximity (ADP), a scenario where the axon of one neuron is in close proximity to the dendrite of another. However, not every ADP leads to the formation of a synapse (see Fig.2). The transformation of an ADP into a functional synapse is a selective process, influenced by a myriad of factors. Understanding the mechanisms and conditions under which ADPs convert into synapses is a significant area of interest in neuroscience. It involves unraveling the complexities of neuronal behavior and connectivity, which are fundamental to comprehending brain function and the underlying processes of various neurological disorders.

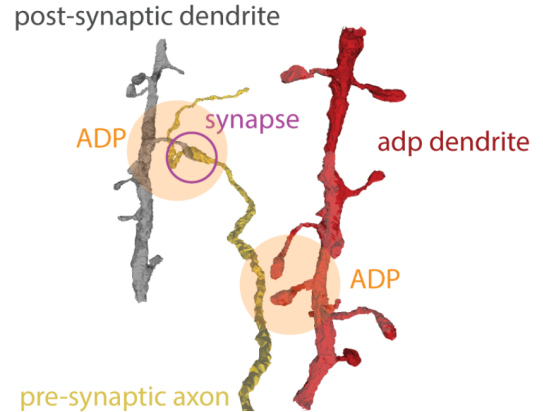


Fig. 2. Examples of cases where some ADPs lead to the formation of synapses (left), while others do not (right). Figure taken from document on Kaggle.

In recent years, the application of machine learning in neuroscience has opened new frontiers in studying these phenomena [2]. Machine learning offers powerful tools to analyze and interpret the vast and complex datasets characteristic of neuronal studies [3]. By leveraging data on each ADP, including both functional data (such as neuronal activity patterns) and morphological data (such as the shape and structure of neurons), machine learning algorithms can predict the

likelihood of synapse formation. The integration of machine learning in the study of neural synapse connections represents a convergence of computational power and biological inquiry.

The task of predicting whether a neural synapse connection will form, based on axonal dendritic proximity (ADP) data, is fundamentally a binary classification problem [4]. In this context, machine learning, particularly supervised learning, plays a pivotal role. Supervised machine learning involves training algorithms on a labeled dataset, where the outcome or 'label' for each data point is known. The algorithm learns to map input data to the corresponding output labels, enabling it to make predictions on new, unseen data. In the case of neural synapse connections, the two classes are the presence or absence of a synapse, making it a quintessential example of binary classification.

In this report, we focus on the application of the Random Forest model, a renowned and powerful classifier, to analyze data derived from Axonal Dendritic Proximity (ADP) and predict the formation of neural synapse connections. The Random Forest algorithm, known for its robustness and accuracy in handling complex classification tasks, is particularly well-suited for this challenge due to its ability to handle large datasets and its inherent mechanisms for reducing overfitting. In our approach, we focused on creating new features from the existing data about neurons. This process, known as feature engineering, involved transforming the original data into new forms that were better suited for making accurate predictions. By doing this, we were able to improve our model's ability to predict neural synapse connections.

The performance of our Random Forest model was rigorously evaluated on various datasets, yielding promising results. On our test set, which constituted 20% of the data, the model achieved a balanced accuracy of 83.56%, demonstrating its strong generalization capabilities. Further validation was obtained through its application to Kaggle's public and private leaderboards, where it achieved balanced accuracies of 72.42% and 77.06%, respectively. These outcomes not only affirm the model's effectiveness in a competitive data science environment but also highlight its consistency across different data scenarios. The success of this approach underscores the potential of machine learning techniques in advancing our understanding of neural connectivity and their broader applications in computational neuroscience.

II. METHOD AND RESULT

A. Data Description

We analysed a comprehensive dataset consisting of 185,832 entries, each representing a potential Axonal Dendritic Proximity (ADP) between neurons. The dataset is structured into 34 columns, encompassing various types of data, including integer (int64), floating-point numbers (float64), boolean (bool), and object (string or mixed types). The total size of the dataset is substantial, with 1,122 features after preprocessing, indicating a rich and detailed collection of information.

The dataset includes several key features:

- **Spatial Coordinates:** Features such as `axonal_coor_x`, `axonal_coor_y`, `axonal_coor_z`, `dendritic_coor_x`, `dendritic_coor_y`, and `dendritic_coor_z` provide the spatial coordinates of axonal and dendritic points, offering insights into their physical positioning.
- **Distances and Scores:** Attributes like `adp_dist`, `post_skeletal_distance_to_soma`, and `pre_skeletal_distance_to_soma` quantify distances relevant to neuronal structures, while `pre_oracle`, `pre_test_score`, `post_oracle`, and `post_test_score` offer various score-based metrics.
- **Receptive Field Coordinates:** `pre_rf_x`, `pre_rf_y`, `post_rf_x`, and `post_rf_y` represent coordinates related to the neurons' receptive fields.
- **Nucleus Information:** Features such as `pre_nucleus_x`, `pre_nucleus_y`, `pre_nucleus_z`, `post_nucleus_x`, `post_nucleus_y`, `post_nucleus_z`, `pre_nucleus_id`, and `post_nucleus_id` provide detailed information about the nuclei of the pre- and post-synaptic neurons.
- **Categorical Data:** Columns like `compartment`, `pre_brain_area`, and `post_brain_area` contain categorical data describing various anatomical and functional classifications.
- **Feature Weights and Morphological Embeddings:** `pre_feature_weights`, `post_feature_weights`, `pre_morph_embeddings`, and `post_morph_embeddings` are complex features that encapsulate advanced morphological and functional characteristics of the neurons.

The target variable, `connected`, is a boolean attribute indicating whether a synapse has formed (True) or not (False). In our dataset, the number of connected instances (synapses) is 1,366, while the number of not connected instances is 184,466, highlighting a significant class imbalance. This imbalance underscores the challenges in predicting rare but crucial events of synapse formation.

B. Data Augmentation

To tackle the issue of unbalanced data, where one class is much more common than the other, we employed a technique known as SMOTE (Synthetic Minority Over-sampling Technique) [5]. This method helps balance our dataset by creating synthetic examples of the less common class, ensuring that both classes are represented more equally. Note that rather than Random Over Sampling, we utilized a SMOTE process, even though we generated datasets for both. As we hoped to optimize the generalizability of our model, we decided that SMOTE would be less prone to overfitting.

The mechanism of SMOTE involves several steps. First, it identifies the minority class in the dataset – the class with fewer instances. For each instance in this minority class, SMOTE finds its k -nearest neighbors (where k is a small integer, such as 5). It then randomly selects one of these neighbors and generates a new, synthetic instance that lies somewhere between the original instance and this selected

neighbor. This is achieved by interpolating between the two instances in the feature space, effectively creating a new instance that is similar, but not identical, to existing instances. The process is repeated until the dataset reaches a more balanced class distribution. These synthetic instances are added to the training dataset, enriching the minority class and providing a more robust basis for training machine learning models.

C. Feature Engineering

1) *Feature Similarity*: One new feature we devised is based on the concept of a neuron's tuning function, which reflects how neurons in the visual cortex respond to various visual stimuli. The 'feature_weights' vectors in the dataset serve as a summary of each neuron's tuning function. Given that neurons with similar tuning functions are more likely to form synapses [], we engineered a feature to quantify this similarity:

$$\text{Feature Similarity} = \frac{\sum(\text{pre_feature_weights} \times \text{post_feature_weights})}{\|\text{pre_feature_weights}\| \times \|\text{post_feature_weights}\|} \quad (1)$$

2) *Distance-Based Features*: We also engineered features based on various types of distances:

- *Nuclear distance*: the Euclidean distance between the nuclei of pre- and post-synaptic neurons.

$$\text{Nucleus Distance} = \sqrt{(x_{\text{post}} - x_{\text{pre}})^2 + (y_{\text{post}} - y_{\text{pre}})^2 + (z_{\text{post}} - z_{\text{pre}})^2} \quad (2)$$

- *Skeletal Distance*: The difference in skeletal distance to the soma between post- and pre-synaptic neurons.

$$\text{Skeletal Distance} = (\text{post-pre})_{\text{skeletal_distance_to_soma}} \quad (3)$$

- *Receptive Field Distance*: the Euclidean distance between the receptive field coordinates of the neurons.

$$\text{RF Distance} = \sqrt{(rRF_{x,\text{post}} - RF_{x,\text{pre}})^2 + (RF_{y,\text{post}} - RF_{y,\text{pre}})^2} \quad (4)$$

3) *Average Scores*: Additionally, we computed average scores for oracle and test scores, assuming that these averages might correlate with synaptic connectivity:

$$\text{Oracle Average} = (\text{pre_oracle} + \text{post_oracle})/2 \quad (5)$$

$$\text{Test Score Average} = (\text{pre_test_score} + \text{post_test_score})/2 \quad (6)$$

4) *Projection Group*: Finally, we introduced a feature to capture the projection regions, which are indicative of the brain regions where the pre-synaptic and post-synaptic neurons are located. This feature, termed 'projection_group', was generated by concatenating the pre- and post-brain area fields. This feature is particularly significant as it encapsulates the

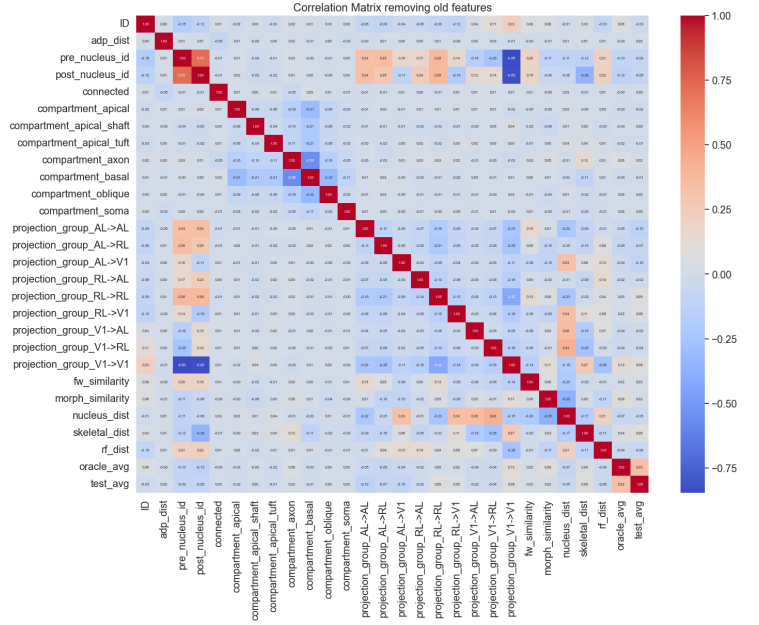


Fig. 3. Correlation Matrix for the dataset with new features

functional relationship between different brain regions, which can be crucial in understanding synaptic connectivity.

$$\text{Projection_Group} = \text{pre_brain_area} \rightarrow \text{post_brain_area} \quad (7)$$

After the construction of new features, we opted to remove the original features that were utilized in their construction. The newly engineered features are expected to contain the essential information from the original features. Retaining the original features alongside the new ones could introduce redundancy into the model. Additionally, we excluded the morphological embedding features from our analysis due to the presence of missing values (N/A), which posed challenges in the data processing and analysis stages.

D. Correlation Matrix

We calculated the correlation matrix for different features. A correlation matrix is a table showing correlation coefficients between variables in Fig.(3). Each cell in the table shows the correlation between two variables. The value is in the range of -1 to 1. If two variables have high correlation (near 1 (red) or -1 (blue)), it suggests a strong relationship between the two, whereas a correlation close to 0 suggests no linear relationship.

In the correlation matrix, we find most features do not have strong linear relationships with one another. Only a few pairs of features have strong correlations. For example, the correlation between projection_group_V1→V1 and pre (post)_nucleus_id is -0.55(-0.53), which is strongly negatively correlated. Such insights are valuable in understanding the interdependencies among the features and guiding further feature selection or engineering processes.

E. Feature Selection

Both feature engineering and the correlation matrix helped guide our feature selection. As our final model was random

forest, where feature selection is less crucial compared to models such as logistic regression, we performed slightly less stringent feature selection than some of our other models. At the same time, we used outside literature to help inform elements of our process. For instance, noting the importance of pathways in neuron synapse formation, we maintained inclusion of the projection group [1] [6]. for future research, we would have liked more detailed data regarding the proteins within the synapse process, given the research we identified regarding how crucial proteins are—both regarding the pathways in which they travel, and the existence of specific proteins such as KIF1A or synaptotagmins which can affect synapse formation [1] [6].

F. Selection of Random Forest Classifier

For our study, we chose the Random Forest Classifier as our primary machine learning model. This decision was influenced by several key attributes of Random Forest that make it particularly suitable for our dataset and objectives [7]. Random Forest, an ensemble learning method, is known for its high accuracy, robustness to overfitting, and ability to handle large datasets with a mix of categorical and numerical features. Its ensemble nature, which involves aggregating predictions from multiple decision trees, helps in achieving a more generalized model performance. In both randomizing the features in addition to the observations at each split, random forest tends to prevent overfitting compared to other machine learning models. This is because the algorithm involves bagging and building a random forest tree at each split. In doing so, the process create many decorrelated trees, which helps with decreasing variance. While each individual random forest tree is a poor predictor on its own, the strength of the model lies in being able to average all the correlated trees. Moreover, Random Forest works well with tabular data, which is the data we were working with. Thus, it works well as an "off-the-shelf" option. With that being said, Random Forest is not a model that works well with extremely imbalanced classes. This was the reasoning for our balancing of the data, as aforementioned.

As we ultimately chose 28 features after our feature engineering process, we selected a tree depth of 5. This is because after searching several different options ranging from 3 to 15, we found that a tree depth of 5 provided us with the best performance on the public leaderboard. This also aligned with our expectations, as the default starting value for tree depth is the square root of the number of total features (i.e., $\sqrt{28}$), as it generally works well for the tree depth hyperparameter. This is because too small of a value leads to underfitting, whereas a high tree depth often leads to overfitting to the training data. We noticed this quite blatantly through our empirical process, as high values for our tree depth decreased our accuracy sometimes by amounts of even 10%. Moreover, we also sought to balance the number of trees based on performance on the public leaderboard.

G. Training Process

The training process was conducted using a pipeline approach in Python's scikit-learn library. The pipeline encapsulated both data preprocessing and model training steps, ensuring a streamlined workflow. The preprocessing step involved scaling the features using StandardScaler, which standardizes the features by removing the mean and scaling to unit variance. This is particularly important for models like Random Forest that are sensitive to the scale of input features.

We utilized the `train_test_split` method to divide our dataset into training and testing sets, with 20% of the data reserved for testing. The model was trained on the resampled dataset using SMOTE to address the class imbalance issue, enhancing its ability to predict the minority class effectively.

Ultimately, we found that the following hyperparameters provided the best results on the Kaggle test sets:

```
RandomForestClassifier
(n_estimators=80,
max_depth=5,
random_state=1)
```

While simple and minimal, this model ended up performing the best out of all of the ones we tested. Both team members tested variations of logistic models utilizing different features; each team member also separately tested and tuned the various following models: AdaBoost, Gradient Boosting, Logistic Regression with PolynomialFeatures, Neural Network (dense hidden layers), and Random Forest. If we had had additional time, we may have also preferred to conduct further model stacking. While our AdaBoost models did perform well on the public leaderboard, we ultimately submitted one AdaBoost model and the Random Forest model discussed here. This was because out of all our Random Forest submissions, the above model performed best on the public leaderboard. We also anticipated that Random Forest might generalize better than AdaBoost, even if two of our AdaBoost model submissions performed better on the public leaderboard. As expected, the Random Forest Model ended up generalizing the best, and while simple, it was our best performer.

Lastly, note that there have been recent developments improving conservation in machine learning with random forest models where models are conserved across multiple runs [8]. Such methods have been found to improve accuracy, and this may be an interesting venue for further research.

H. Evaluation Metrics

In evaluating the performance of our Random Forest model, we focused on three key metrics: the confusion matrix, accuracy, and balanced accuracy. Each of these metrics provides valuable insights into different aspects of the model's performance.

The confusion matrix is a table used to describe the performance of a classification model on a set of test data for which the true values are known. It allows us to visualize the model's predictions in four categories: True Positives (TP),

	Predicted Positive	Predicted Negative
Actual Positive	TP	FN
Actual Negative	FP	TN

TABLE I
CONFUSION MATRIX

True Negatives (TN), False Positives (FP), and False Negatives (FN). The confusion matrix form is represented in Table.(I).

- True Positives (TP): Correctly predicted positive observations.
- True Negatives (TN): Correctly predicted negative observations.
- False Positives (FP): Incorrectly predicted positive observations.
- False Negatives (FN): Incorrectly predicted negative observations.

Accuracy is the ratio of correctly predicted observations to the total observations. It is suitable for symmetric datasets where values of false positives and false negatives are almost the same. The formula for accuracy is:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (8)$$

Balanced accuracy is used when dealing with imbalanced datasets. It is defined as the average of recall obtained on each class. It accounts for both the true positive rate (sensitivity) and the true negative rate (specificity), providing a more balanced evaluation metric for imbalanced datasets. The formula for balanced accuracy is:

$$\text{Balanced Accuracy} = \frac{1}{2} \left(\frac{TP}{TP + FN} + \frac{TN}{TN + FP} \right) \quad (9)$$

I. Prediction

The Random Forest model was evaluated on a test set comprising 20% of our data, following the application of SMOTE to address class imbalance. The confusion matrix for the test set predictions is as follows: The model achieved an

	Predicted No Synapse	Predicted Synapse
Actual No Synapse	27269	9705
Actual Synapse	2426	34387

TABLE II
CONFUSION MATRIX FOR TEST SET

accuracy of approximately 83.56%, and balanced accuracy of nearly 83.58% on this test set. This high level of accuracy indicates that the model was able to correctly predict the presence or absence of synapse formation in a significant majority of cases.

Further validation of our model's performance was obtained through its application to Kaggle's public and private leaderboards. On the public leaderboard, the model achieved a balanced accuracy of 72.42%, while on the private leaderboard, it reached a balanced accuracy of 77.06%. These results demonstrate the model's robustness and its ability to generalize well to external datasets. The slight variation in performance

between the public and private leaderboards could be attributed to differences in the underlying data distributions or the specific characteristics of the cases included in each set.

J. Interpretability

The high accuracy and balanced accuracy on the test set, coupled with solid performance on the Kaggle leaderboards, underscore the effectiveness of our Random Forest model in predicting neural synapse connections. The results from the confusion matrix further reinforce the model's capability to distinguish between synapse formation and non-formation scenarios accurately. These findings highlight the potential of machine learning approaches, particularly ensemble methods like Random Forest, in advancing our understanding of complex biological phenomena such as neuronal connectivity.

III. DISCUSSION

We want to stress several points: first, we did not use a separate validation set for hyperparameter tuning, particularly for parameters such as the number of trees (`n_estimators`) and the maximum depth (`max_depth`) in our Random Forest model. This decision was based on the ensemble nature of Random Forest, which tends to generalize well over different datasets. However, we acknowledge that incorporating a validation step, such as k-fold cross-validation, could further strengthen the model by providing insights into its performance on unseen data and helping in hyperparameter optimization. It is relevant to note that we did attempt this, but found disparities between our own self-defined test error using the training data and the public leaderboard. Regarding the hyperparameter of number of trees, the goal was to choose the number of trees for which out-of-bag error stabilized, to balance between computation power and decreased bias. At the same time, our team was somewhat limited in terms of computation power given our resources. Moreover, somewhat counterintuitively, we found that more trees did not necessarily improve our performance on the public leaderboard, while it did within our self-determined test set (that we used to validate our models). Consequently, while we tried values ranging from 20 to 400, we ultimately landed on 80 as producing the best results from our candidate values for this hyperparameter. This was not only through validating our performance, but also by submitting on the public leaderboard and focusing on our Kaggle results.

Note that one aspect we could have improved on was application of SMOTE. We applied it to our overall training data as we sought to train the models for the Kaggle test sets; however, in doing so, this affected our internal validation. Due to our code's structure, we applied SMOTE across the entirety of our training data. However, in doing so, our internal test set ended up being not as reliable. This was an insight that we did not identify until too late, unfortunately, but is valuable to internalize and understand for our future endeavors. Furthermore, it's important to note that during our hyperparameter tuning process, specifically adjusting the number of trees and maximum depth, we observed a tendency towards overfitting. This was evidenced by lower scores on the Kaggle

leaderboard, a phenomenon that was not apparent in the validation within the original dataset. This suggests that while Random Forest is generally robust, careful attention must still be paid to hyperparameter settings to avoid overfitting, especially in a competitive modeling environment like Kaggle.

Second is the convergence of accuracy and balanced accuracy metrics, which we attribute to the application of the SMOTE technique. By artificially balancing the class distribution in our dataset, SMOTE effectively equalized the influence of each class on these metrics. In the context of our originally imbalanced dataset, accuracy alone could have been misleading, disproportionately reflecting the majority class's prevalence. However, the use of SMOTE ensured that both accuracy and balanced accuracy provided a more truthful representation of the model's predictive capabilities across both classes.

Our study, while comprehensive, has limitations that could be addressed in future research. For instance, exploring a wider range of hyperparameters or incorporating additional features could potentially improve model performance. Additionally, implementing a validation phase in future iterations could provide deeper insights into the model's generalizability and robustness.

Moreover, this project has been extremely interesting within the greater context of neuroscience and machine learning. For instance, we were initially quite interested in neural net models. This is due to existing literature that has showed its potential, oftentimes modeled on our biological neurons. Researchers at MIT CSAIL developed a "'closed-form continuous-time' (CfC) neural network," a form of liquid neural network inspired by the brains of small species with significant improvements in computational speed [9]. This is a form of a "liquid" neural network in general, in which networks "fluidly" adjust their stacked models and equations to new data, rather than being fixed after training. Ramin Hasani, one of the researchers and proponents of liquid neural networks, said he analyzed how *C. elegans* neurons behave, and gained inspiration from them [10]. Such machine learning inspiration from neuroscience is incredibly common and widespread within machine learning advances. For instance, researchers from Tsinghua University in China recently published a paper incorporating elements of continual learning and learning plasticity within machine models [11]. In particular, existing machine learning models typically try to maintain memory stability to avoid "catastrophic forgetting," but such methods are typically less flexible than biological systems to incorporating new changes. The researchers sought to utilize inspiration from a fruit fly neurological system, from the *Drosophila* genus, to improve upon learning plasticity within machine learning models. Some more recent developments of note within the life sciences and machine learning space include several studies utilizing novel machine learning methods to analyze patterns relating to proteins, which is not directly related to neuron synapse formation but may provide valuable indirect insights [12] [13] [14].

It is quite fascinating to see the interplay between neu-

rosience and machine learning, where neurological phenomena inspire new machine learning models, yet machine learning models conversely help us gain further insight into unexplained neurological phenomena. Ultimately, this project represented a baby step for us as students into the exploration and interplay between neuroscience and machine learning, and provided a valuable and rewarding experience.

IV. CONCLUSION

In conclusion, our study demonstrates the application of Random Forest in predicting neural synapse formation from ADP data, highlighting the critical role of feature engineering and class imbalance correction. The results achieved on both the test set and Kaggle leaderboards underscore the model's effectiveness and its potential utility in computational neuroscience. This project not only deepens our understanding of machine learning techniques but also opens avenues for further research in the intricate field of neural connectivity.

ACKNOWLEDGMENT

Within the project, we worked together to feature engineer. We then utilized our engineered features to separately test the following models: Logistic, AdaBoost, Random Forest, Gradient Boosting, and Neural Networks. Both team members tuned variations on the logistic model, while Grace tuned Random Forest and AdaBoost methods and Yiming tuned neural networks and gradient boosting methods. Yiming also primarily authored the majority of the report, with Grace writing the Feature Selection, Random Forest, and partially the Training Process subsections and Discussion.

We express our sincere thanks to Prof. Genevera Allen for her exceptional guidance in the machine learning course (ELEC 478/578), which greatly enriched our understanding and application of the subject. This report, developed as our final project, is a testament to the practical and theoretical knowledge gained under her instruction. We also extend our gratitude to our peers and teaching assistants for their invaluable support throughout this enriching academic journey.

REFERENCES

- [1] S. Oßwald, "Deciphering neurons: A journey into synapse formation," *Neuroscience News*, 2023.
- [2] M.-A. T. Vu, T. Adalı, D. Ba, G. Buzsáki, D. Carlson, K. Heller, C. Liston, C. Rudin, V. S. Sohal, A. S. Widge *et al.*, "A shared vision for machine learning in neuroscience," *Journal of Neuroscience*, vol. 38, no. 7, pp. 1601–1607, 2018.
- [3] K. P. Murphy, *Machine learning: a probabilistic perspective*. MIT press, 2012.
- [4] R. Kumari and S. K. Srivastava, "Machine learning: A review on binary classification," *International Journal of Computer Applications*, vol. 160, no. 7, 2017.
- [5] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "Smote: synthetic minority over-sampling technique," *Journal of artificial intelligence research*, vol. 16, pp. 321–357, 2002.
- [6] T. C. Südhof and R. C. Malenka, "Deciphering neurons: A journey into synapse formation," *Neuron*, vol. 60, pp. 469–476, 2008.
- [7] M. M. G. . E. E. Khaled Fawagreh, "Random forests: from early developments to recent advancements," *Systems Science Control Engineering*, vol. 2, pp. 602–609, 2014.
- [8] M. Sipper and J. Moore, "Conservation machine learning: a case study of random forests," *Scientific Reports*, vol. 11, p. 3629, 2021.

- [9] R. Hasani, M. Lechner, A. Amini, L. Liebenwein, A. Ray, M. Tschaikowski, G. Teschl, and D. Rus, "Closed-form continuous-time neural networks," *Nature Machine Intelligence*, vol. 4, no. 11, pp. 992–1003, 2022.
- [10] R. Hasani, M. Lechner, A. Amini, D. Rus, and R. Grosu, "Liquid time-constant networks," vol. 35, no. 9, pp. 7657–7666, 2021.
- [11] Z. X. L. Q. e. a. Wang, L., "Incorporating neuro-inspired adaptability for continual learning in artificial intelligence," *Nature Machine Intelligence*, 2023.
- [12] P. G. . B. J. Mastropietro, A., "Learning characteristics of graph neural networks predicting protein–ligand affinities," *Nature Machine Intelligence*, 2023. [Online]. Available: <https://doi.org/10.1038/s42256-023-00756-9>
- [13] W. F. L. L. e. a. Fang, X., "A method for multiple-sequence-alignment-free protein structure prediction using a protein language model," *Nature Machine Intelligence*, 2023. [Online]. Available: <https://doi.org/10.1038/s42256-023-00721-6>
- [14] T. H. L. H. e. a. Lin, P., "Protein–protein contact prediction by geometric triangle-aware protein language models," *Nature Machine Intelligence*, vol. 5, 2023. [Online]. Available: <https://doi.org/10.1038/s42256-023-00741-2>